

STAX — Combined Documentation

Daily Game Development Series, Day 7 — Whitepaper · History · Source Code Detail

Project	STAX
Series	Daily Local Game Development Series — Day 7
Author	Mayank Swaraj
Brand	Mayank Creations
GitHub	github.com/mayankswaraj18cr-cmd
LinkedIn	linkedin.com/in/mayank-swaraj-4b84b6410
Stack	Vanilla JavaScript, HTML5 Canvas, Web Audio API
Document Type	Combined Whitepaper + History + Source Code Detail

WHITEPAPER

1. Overview

STAX is Day 7 of the daily vanilla JavaScript game development series under Mayank Creations. It is a precision block-stacking game: a sliding block oscillates horizontally above the tower and the player taps once to drop it. The overlapping portion with the block below forms the new top; the overhanging portion is discarded. A shrinking stack width and a miss condition end the run.

Series position: Days 1–6 covered grid puzzle, vertical dodge, gravity runner, brick-breaker, snake, and space shooter. Day 7 is the first entry driven entirely by one-shot timing precision with no directional input — the player's only agency is the moment of the tap.

2. Design Goals

- Single-event input model: the entire game is played with repeated single taps — no held input, no directional control, no continuous tracking.
- Perfect-placement reward: a tolerance window awards a combo multiplier and preserves full block width, giving skilled players a compounding score advantage.
- Progressive speed scaling: slider velocity increases by 0.08 per block placed, so early floors are forgiving and late towers demand frame-precise timing.

- Viewport scrolling: the canvas follows the tower upward, keeping the active drop zone in the upper third of the screen regardless of height.
- Minimum-width kill condition: a stack narrower than 18px ends the game, preventing indefinitely thin slivers from trivialising height accumulation.

3. Core Mechanics

Each block has a world-space y coordinate that decrements by `BLOCK_H` per placement. The camera offset (`scrollY`) increases to keep the active layer in the upper viewport region. All draw calls subtract `scrollY` from the world y coordinate to produce screen coordinates, implementing a simple 2D camera.

On drop: the overlap between slider x-range and top-block x-range is computed. Zero overlap triggers game over. Non-zero overlap within `PERFECT_TOL` pixels of exact alignment awards perfect placement and preserves width. Otherwise the landed block width equals the overlap width and combo resets.

4. Scoring

Each placement scores 10 base points. Perfect placements additionally score $5 \times$ current combo, compounding with consecutive perfects. A combo counter resets on any non-perfect placement. Score is displayed live; best score persists for the session.

5. Audio Design

- Drop: ascending sine series ($220 + \text{level} \times 18$ Hz) — pitch rises with height, giving auditory feedback of tower growth.
- Perfect: two-tone ascending sine ($880 \rightarrow 1100$ Hz) at 70ms spacing, distinctly celebratory versus a standard drop.
- Chop: low sawtooth for the discarded overhang — reinforces the 'cut' metaphor.
- Collapse: descending sawtooth two-note, consistent with series death-sound language.

HISTORY PAPER

1. Position in the Series

STAX is Day 7. It is the first entry in the series with no directional input — the player has no lateral or vertical agency, only timing. This is a deliberate contrast with Day 6 (PULSAR), which had the most complex input model to date (drag positioning + auto-fire + power-up collection).

2. Build Decisions

World-space coordinates for the stack with a single scrollY offset was chosen over recalculating block positions relative to the viewport on every frame. This keeps all game logic in world space and isolates the camera to a single subtraction in the draw function.

The perfect-tolerance window (4px) was calibrated to feel achievable but not trivial at the default starting speed. At maximum speed the slider crosses 4px in roughly 9ms — approximately one frame — making late-game perfects genuinely skilled.

Slider bounce (reversing direction on canvas edge rather than off-screen wrap) ensures the player always has a predictable crossing frequency and can anticipate the return pass.

3. Divergence from Prior Entries

- First entry with a persistent-state 2D camera (scrollY viewport scroll).
- First entry where game difficulty is entirely encoded in one scalar (slider speed) rather than object count, spawn rate, or formation logic.
- First entry with a compounding score multiplier tied to consecutive perfect-precision events rather than collectibles or wave number.
- Fewest distinct mechanics of any entry — intentional; the series tests minimal-mechanic depth against multi-mechanic complexity across different days.

SOURCE CODE DETAIL

1. Overlap and Perfect Detection

```
var overlapL = Math.max(slider.x, top.x);
var overlapR = Math.min(slider.x + slider.w, top.x + top.w);
var overlapW = overlapR - overlapL;

if (overlapW <= 0) { endGame(); return; }

var perfect = Math.abs(slider.x - top.x) <= PERFECT_TOL;
if (perfect) {
  combo++;
  score += 10 + combo * 5;
  stack.push({ x: top.x, y: slider.y, w: top.w, col: slider.col });
} else {
  combo = 0;
  score += 10;
  stack.push({ x: overlapL, y: slider.y, w: overlapW, col: slider.col });
}
```

2. Camera / Viewport Scroll

```
// After each placement:
var screenTop = topBlock.y - scrollY;
if (screenTop < H * 0.4)
  scrollY = topBlock.y - H * 0.4;

// In draw:
function getBlockScreenY(worldY) { return worldY - scrollY; }
```

3. Speed Progression

```
var SPEED_BASE = 2.4, SPEED_INC = 0.08, SPEED_MAX = 7.2;

function spawnSlider() {
  var level = stack.length;
  var sp = Math.min(SPEED_MAX, SPEED_BASE + level * SPEED_INC);
  var fromLeft = level % 2 === 1;
  slider = {
    x: fromLeft ? -top.w : W,
    vx: fromLeft ? sp : -sp,
    ...
  };
}
```

4. Slider Bounce

```
slider.x += slider.vx * (dt / 16);
if (slider.x + slider.w > W && slider.vx > 0) {
  slider.vx *= -1; slider.x = W - slider.w;
}
if (slider.x < 0 && slider.vx < 0) {
  slider.vx *= -1; slider.x = 0;
}
```

5. Minimum Width Kill

```
if (stack[stack.length - 1].w < MIN_W) {
  sndDead(); endGame(); return;
}
```

6. Known Constraints

- scrollY only scrolls upward; no downward camera movement — falling debris is not tracked visually.
- Overhang chop particle burst uses the overhang centroid ($chX + chW/2$), which is visually approximate for very small overhangs.
- Best score is session-only.

"Every system I build is a brick in a larger, self-taught architecture of independent AI work."

- Mayank Swaraj